© 2025 ANSYS, Inc. or affiliated companies
Unauthorized use, distribution, or duplication prohibited.

PySam

ANSYS, Inc.
Southpointe
2600 Ansys Drive
Canonsburg, PA 15317
ansysinfo@ansys.com
http://www.ansys.com
(T) 724-746-3304
(F) 724-514-9494

Jul 11, 2025

ANSYS, Inc. and ANSYS Europe, Ltd. are UL registered ISO 9001:2015 companies.

CONTENTS

1 Getting started 3

2 User guide 5

3 Documentation 21

4 Examples 31

5 Release notes 43

PySam provides a Python scripting interface for SysML2 models. You can load models from any SysML2 tools who provide an implementation of the standard API. The loaded model is mapped into Python object. So you can manipulate models, browse through them, edit them, and push the modification inside your modeling tool.

This library intends to work first with SAM, the SysML2 modeling tool of Ansys.

CHAPTER

ONE

GETTING STARTED

This section's scope is about user installation.

1.1 Installation

```
python -m pip install ansys_sam_sysml2-<version>-py3-none-any.whl
```

1.2 Use

> **Note**
>
> You can find examples *here*.

1.2.1 Connect to the tool

Here, the AnsysSysML2 API Connector is used. If you want to connect to any other SysMLV2 tool, please read the developer guide.

Listing 1: Import the Connector and the Project manager

```
from ansys.sam.sysml2 import AnsysSysML2APIConnector, SysML2ProjectManager
```

The connector provides the project manager with access to the model.

Then create the connector :

Listing 2: Connect to the SysML2 API server

```
ansyssysml2apiconnector = AnsysSysML2APIConnector(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    organization_id="<Orga ID>",  # The Organization ID
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # If the server has a valid SSL
)
```

Then, you can create the project manager and load a project:

Listing 3: Create the project manager and load a project

```
project_manager = SysML2ProjectManager(connector=ansyssysml2apiconnector)

my_bike_project = project_manager.get_project("<Bike Project ID>")
```

CHAPTER

TWO

USER GUIDE

These sections provide help for users of this library.

2.1 Use a connector

A connector is the interface between the library and the SysML V2 Server.

2.1.1 Connectors

> **Note**
>
> To find all required data (Ids, token,..) see *Here*.
>
> Also, you can find classes specification *Here*.

Ansys SysML2 API Connector

It uses the Standard API to load and publish information in a model.

Listing 1: Connect to the SysML2 API server

```
ansyssysml2apiconnector = AnsysSysML2APIConnector(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    organization_id="<Orga ID>",  # The Organization ID
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # If the server has a valid SSL
)
```

Sam Rest API Connector

It uses the SAM REST API to retrieve diagrams in the model.

Listing 2: Connect to the SAM REST API server

```
sam_rest_api_connector = SamRestApiConnector(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # If the server has a valid SSL
)
```

2.2 Load a model

To load a model, you need an instance of a SysML Connector (see *here*).

After this, you can create a SysML2ProjectManager. This class helps you load a model in Python, using the SysML V2 Standard API.

Listing 3: Create SysML2 Project Manager

```
project_manager = SysML2ProjectManager(connector=ansyssysml2apiconnector)
```

You can find information about this class *here*.

With this project manager, you can load your project like this:

Listing 4: Get the Project using Project ID

```
project = project_manager.get_project(
    "<Computer Project ID>"
)  # You can find it in the URL of the Editor
```

Once you got this project variable, you have a Python version of your model.

2.3 Read your model

One of the goal of the lib is to be able to read and parse the model through a Python script.

The loaded model is stored in a *Project*.

2.3.1 Access methods

To parse the project, you can use different access methods.

Dot access

Using the dot access, you can access all direct named element of your SysML element. Also, you can access to some useful top level function, like *get_value* for example.

Sub elements

The script attaches all elements contained in `ownedElement` and `_inheritedFeature`, who has a name, to the container element.

```
>>> myPart.subPart
<class ParUsage>
```

Warning

Limitations of dot access for sub-elements:

- **Duplicate names**: If multiple elements have the same name, only one is accessible through dot access. You cannot differentiate between elements with identical names using this method.
- **Names with spaces**: Elements with spaces in their names (for example, "bike frame") cannot be accessed using dot notation. Python identifiers cannot contain spaces.

Function `get_value`

> **Warning**
>
> This function is only for SysML Features Element.

This is a top level function to help user to get the Value of the feature without having to read the internal structure.

```
>>> myExpressionFeature.get_value()
(10, 'kg')
>>> myIntFeature.get_value()
10
>>> myStringFeature.get_value()
"Hello"
>>> myBoolFeature.get_value()
False
>>> myFloatFeature.get_value()
10.56
```

This function supports:

- All primitive types: LiteralInteger, LiteralString, etc. - Returns the value directly
- Simple expressions: <value> [<unit>] - Returns a tuple: (<value>,<unit short name>)

Otherwise, it raises an exception.

2.3.2 Underscore access

Using the _ access, you can find all SysML methods:

```
>>> myPart._name
myPart
>>> myItem._ownedElement
[<class portUsage>,...]
```

> **Warning**
>
> In this first version, only existing fields (with data) are linked, so it's possible to don't find a function, which exist in SysML V2

2.4 Write in your model

> **Warning**
>
> This feature is in beta version and could have some issues.

2.4.1 Update feature value

There are two functions to update the value of a feature:

- set_value()

- parse_and_set_value()

Function `set_value`

This function is for all primitive types:

```
>>> myFeature.set_value(True)
>>> myFeature.get_value()
True
>>> myFeature.set_value(10)
>>> myFeature.get_value()
10
>>> myFeature.set_value("Hello")
>>> myFeature.get_value()
Hello
>>> myFeature.set_value(10.5)
>>> myFeature.get_value()
10.5
```

The model is updated after all set, to keep it as accurate as possible.

Function `parse_and_set_value`

This function is for more complex expressions:

```
>>> myFeature.parse_and_set_value("10 [m]")
>>> myFeature.get_value()
(10,"m")
>>> myFeature.parse_and_set_value("2 + 10 [kg]")
>>> myFeature.get_value()
Exception UnsupportedValueExpression raised
```

2.4.2 Create new elements

You can create new elements in your model using the `Factory` class.

> **Tip**
>
> A complete example is available *here*.

Listing 5: Create a Factory instance

```
factory = Factory(project, ansyssysml2apiconnector)
```

Then, use the `create_element()` method to create a new model element. You must provide the type of the element, as well as any number of keyword arguments representing its attributes:

```
new_attribute_usage = factory.create_element(
    element_type="AttributeUsage",
    name="new_attribute_usage",
)
```

This creates a new `AttributeUsage` element at the root of your project. The `create_element()` method returns the newly created element.

Listing 6: Create a new AttributeUsage element with owner

```
new_bicycle_frame_length = factory.create_elements(
    element_type="AttributeUsage", name="length", owner=bike.frame
)
```

This creates a new `AttributeUsage` element with the given attributes inside the `Bike` frame.

Note

The list of accepted attributes depends on the type of element you are creating. For example `name`, `owner`, `shortName`, and others defined by the `metamodel`.

You can also assign a value directly when creating the element. There are two ways:

- Use `value=...` for simple values (e.g., numbers).
- Use `expression="..."` for values with units or expressions.

```
new_bicycle_frame_length_with_value = factory.create_element(
    element_type="AttributeUsage",
    name="lengthWithValue",
    owner=bike.frame,
    value=60
)

new_bicycle_frame_length_with_expression = factory.create_element(
    element_type="AttributeUsage",
    name="lengthWithExpression",
    owner=bike.frame,
    expression="60 [cm]"
)
```

This lets you set values directly at creation time, depending on your data format.

Direct attribute modification

You can also make modifications directly to attributes using a simple assignment. This is particularly useful for changing properties like names.

```
>>> my_attribute = factory.create_element(element_type="Attribute", name="OriginalName")
>>> my_attribute._name = "New Name"
>>> print(my_attribute._name)
New Name
```

This allows for quick and direct updates to element properties.

2.5 Working with diagrams

Warning

The functionalities described in this section (loading, downloading, and navigating diagrams) are **only available for projects of type SAM**.

This section describes how to **load**, **download**, and **navigate** diagrams from your SysML v2 project using the SAM REST API.

2.6 Load diagrams

Before interacting with diagrams, they must be loaded using the *SAMDiagramManager* context manager which needs a *SamRestApiConnector*.

2.6.1 Basic usage

First, create a SAM REST API Connector:

Listing 7: Create SAM REST API Connector

```
sam_rest_api_connector = SamRestApiConnector(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # If the server has a valid SSL
)
```

To load diagrams from a project and make them available for further operations like downloading:

Listing 8: Load diagrams using SAMDiagramManager

```
with SAMDiagramManager(connector=sam_rest_api_connector) as diagrams:
    diagrams.load_diagrams(model=project)
```

Outside the `with` block, the `diagram` manager is no longer active. This ensures proper setup and cleanup of resources when working with diagrams.

2.7 Download diagrams

Diagrams must be loaded inside a *SAMDiagramManager* context before they can be downloaded.

Also, instantiate a *SamDiagramDownloader*:

Listing 9: Create a SamDiagramDownloader

```
downloader = SamDiagramDownloader(connector=sam_rest_api_connector, project_id=project._
↪id)
```

2.7.1 Download all diagrams

Once the diagrams are loaded, you can download **all diagrams** and save them into a zip archive:

Listing 10: Download all diagrams with custom parameters

```
path = downloader.download_all_diagrams(
    path="C:/Diagrams/Images/",
    file_format="jpeg",
    filename="download_all_diagrams_with_args.zip",
)
print(f"ZIP saved at: {path}\n")
```

Default Parameters:

- `file_format`: "SVG"
- `filename`: "<PackageName>_<FileFormat>_diagrams.zip"

If not specified, these defaults are used to name and format your download archive.

2.7.2 Download a single diagram

You can also download a specific diagram in a given format:

Listing 11: Get desired Diagram

```
first_diagram = project.get_root_package().__diagram[0]
first_diagram_id = first_diagram._id
```

Listing 12: Download a single diagram

```
path = downloader.download_diagram(
    diagram_id=first_diagram_id,
    path="C:/Diagrams/Images/",
    file_format="svg",
)
print(f"Diagram saved at: {path}")
```

2.7.3 Download diagrams in a loop

You can iterate through diagrams inside a specific section of your model, for example, *Usage*:

Listing 13: Get desired Diagrams

```
usage_diagrams = project.get_root_package().Usage.__diagram
```

Listing 14: Download diagrams from Usage section in a loop

```
for i, diagram in enumerate(usage_diagrams, 1):
    downloader.download_diagram(
        diagram_id=diagram._id, file_format="png", path="C:/Diagrams/Images/Usage"
    )
    print(f"> Saved Usage diagram #{i}: {diagram._plane._model_element._name}")
```

2.8 Navigate model from diagrams

Each diagram object links to a model element, and you can access its name or other metadata:

2.8.1 Get diagram metadata

Listing 15: Get desired Diagram

```
usage_diagrams = project.get_root_package().Usage.__diagram
```

Listing 16: Get diagram metadata from model element

```
print(first_diagram._plane._model_element._name)
```

This returns the name of the associated model element the diagram represents.

2.8.2 Loop through diagrams

To print out names of diagrams from a section of the model:

Listing 17: Get desired Diagrams

```
usage_diagrams = project.get_root_package().Usage.__diagram
```

Listing 18: Loop through diagrams and print names

```
for diagram in usage_diagrams:
    print("Diagram name:", diagram._name)
```

> **Note**
>
> Navigating through `_plane` and `_model_element` and `_owned_diagram_elements` allows you to discover the logical elements the diagram is tied to.

2.9 Finding information

This section contains instructions for finding required information such as the Project ID or your Auth token.

2.9.1 Find project ID

You can find the Project ID in different places:

- **Editor**: Open the project in the editor; the Project ID appears in the web page URL:

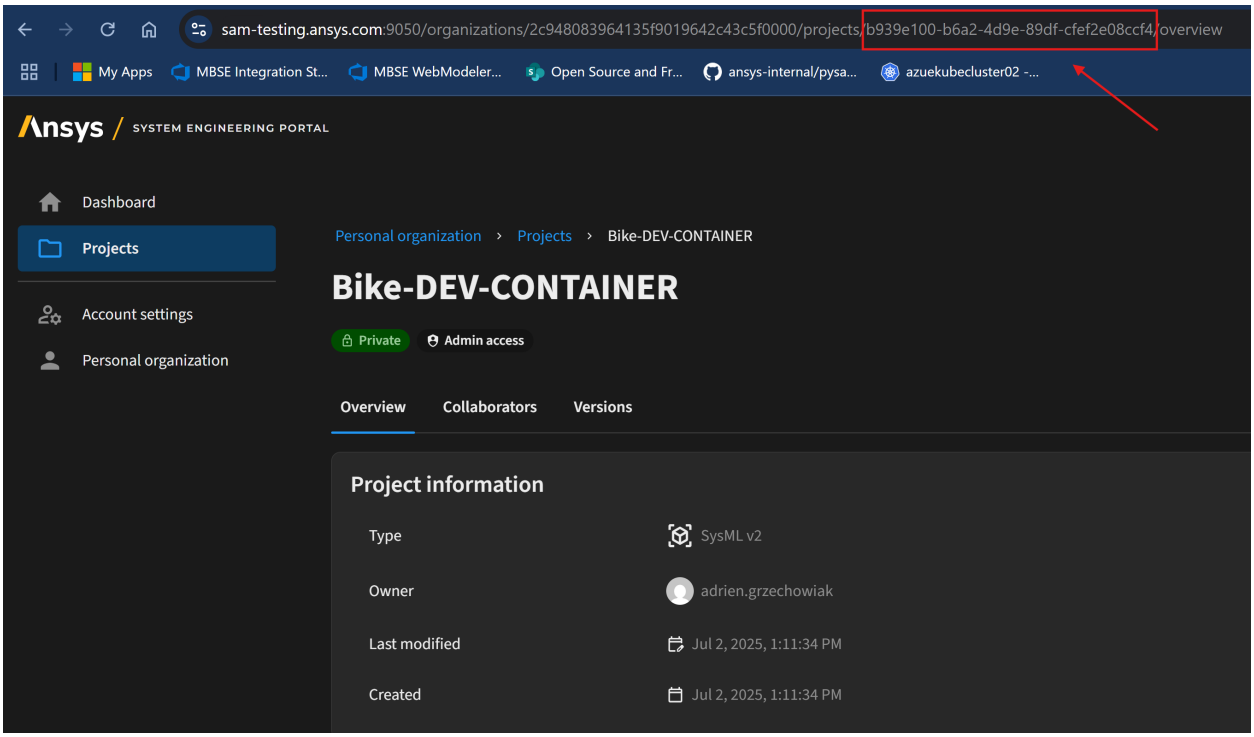
- **Dashboard**: Open the project view page; the Project ID appears in the web page URL:

2.9.2 Find organization ID

To get your Organization ID, follow these steps:

- Go to your Dashboard
- Click your Organization:
- You can find your Organization ID in the URL of the page:

You have successfully retrieved your Organization ID.



2.9.3 Find bearer token

You can use a temporary token for quick access or create a Personal Access Token (PAT) for longer-term use.

Temporary token

To get your Bearer Token, open the SAM Editor Dashboard on your browser. Then click `F12` to open the browser's console.

Next, go to the *Network* tab and reload the page.

Many requests appear. Search for one of these:

- `me`
- `organizations`
- `access-rights`

Then, click the relevant request and go to the *Headers* tab.

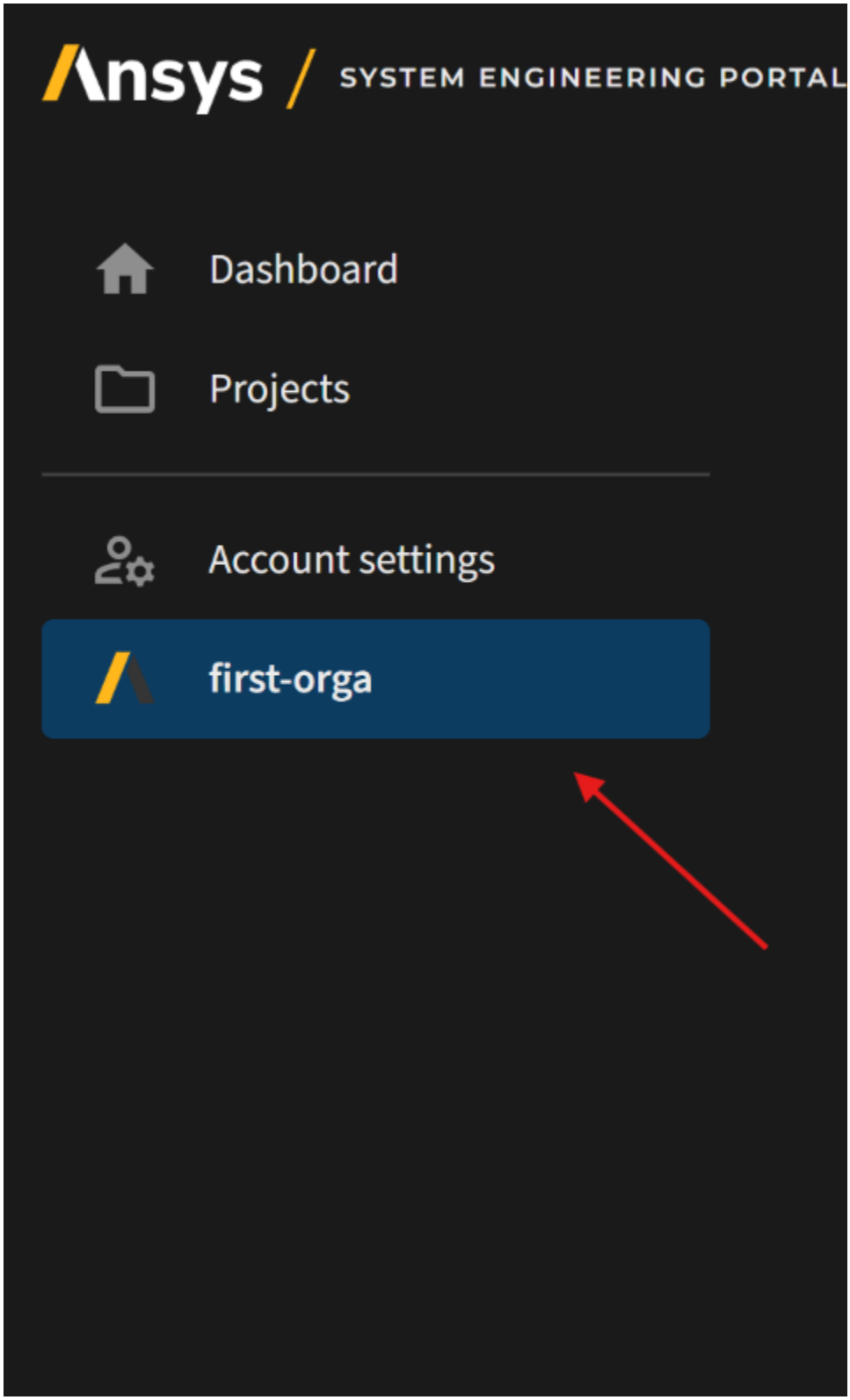
The *Authorization* header shows the value of your Bearer Token.

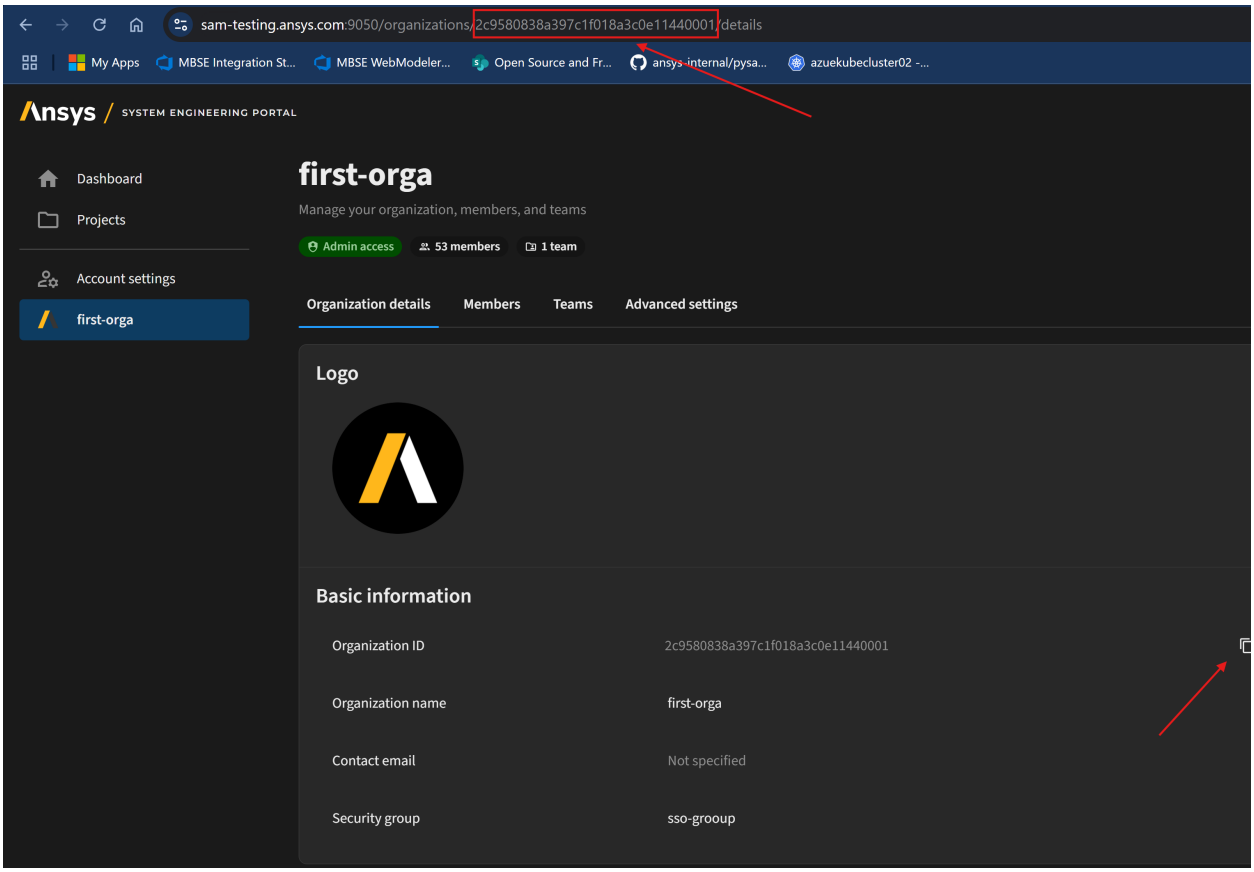
> **Warning**
>
> Only take the Token, after "Bearer "

Personal access token (PAT)

Creating a Personal Access Token (PAT) is the recommended method for authentication with PySAM due to its longer validity.

New Method (Recommended for Current Servers)


Ansys
SYSTEM ENGINEERING PORTAL
Dashboard
Projects
Account settings
first-orga



Follow these steps to generate a PAT directly from your account settings:

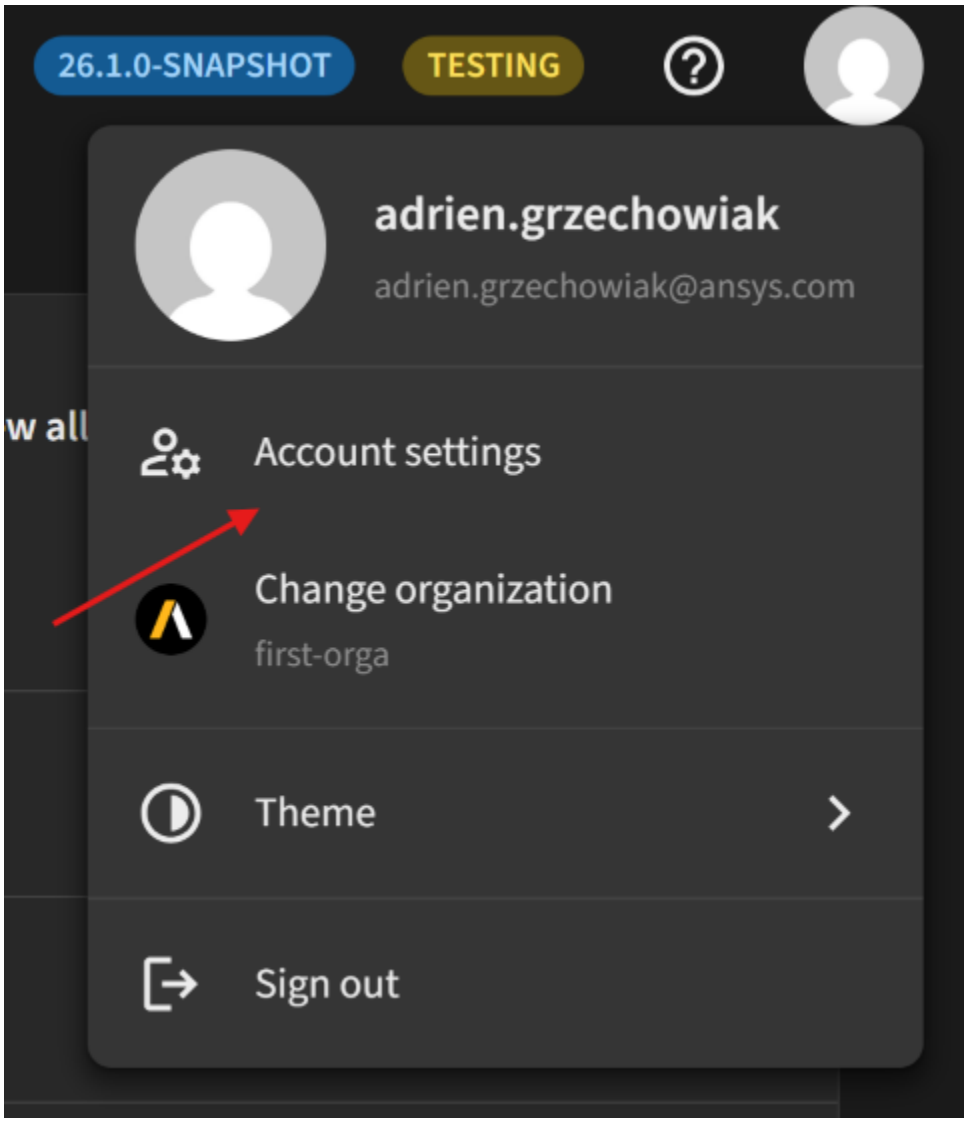
1. **Access Account Settings:** From the main page of the SAM Editor, click your account logo in the top right corner, then select "Account settings."
2. **Navigate to Personal Access Tokens:** On the settings page, click "Personal access tokens" in the navigation pane.
3. **Generate New Token:** Click the "Generate new token" button.
4. **Configure Token Details:** Enter a descriptive label for your token in the "Token Label" field and set the desired expiration date. Then, click "Generate Token."
5. **Copy Generated Token:** Your newly generated token displays. Copy this token immediately.

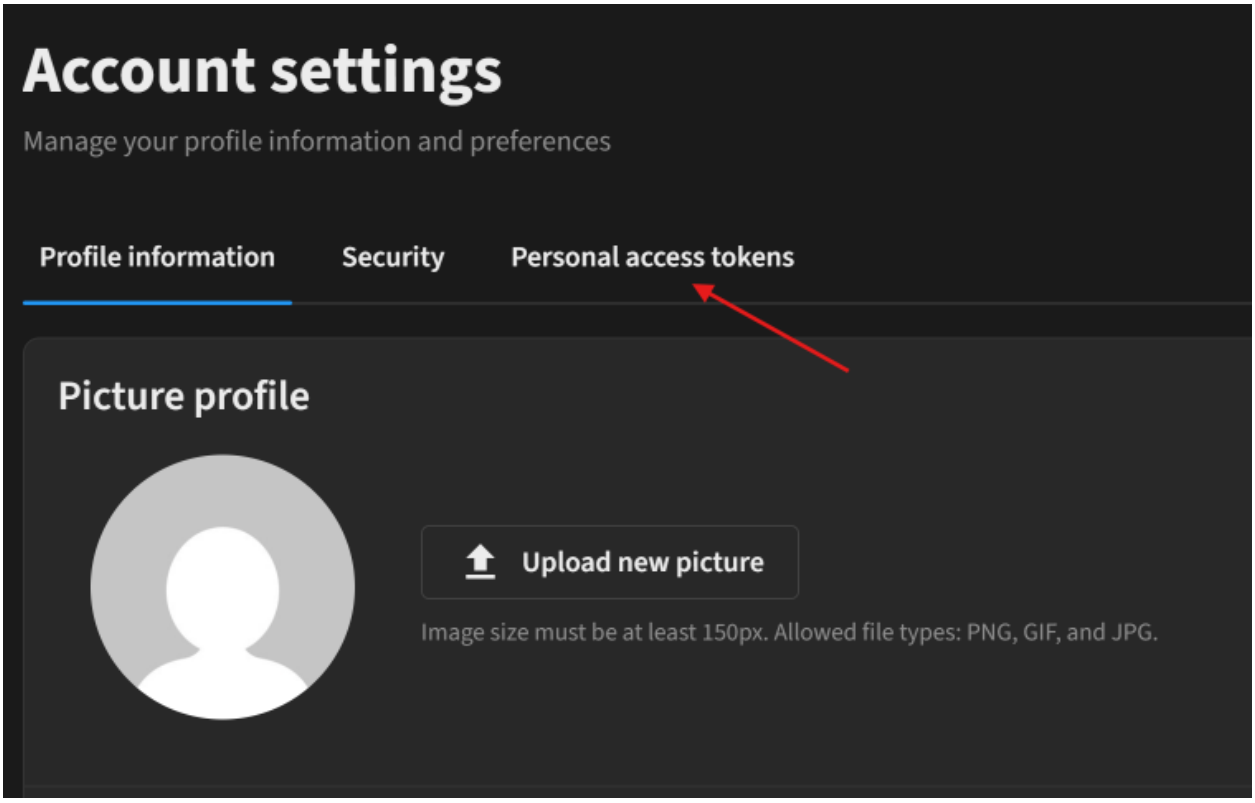
> **Important**
>
> **Store this token in a secured location.** For security reasons, it is not displayed again once you leave this page. You can always generate a new one if needed.

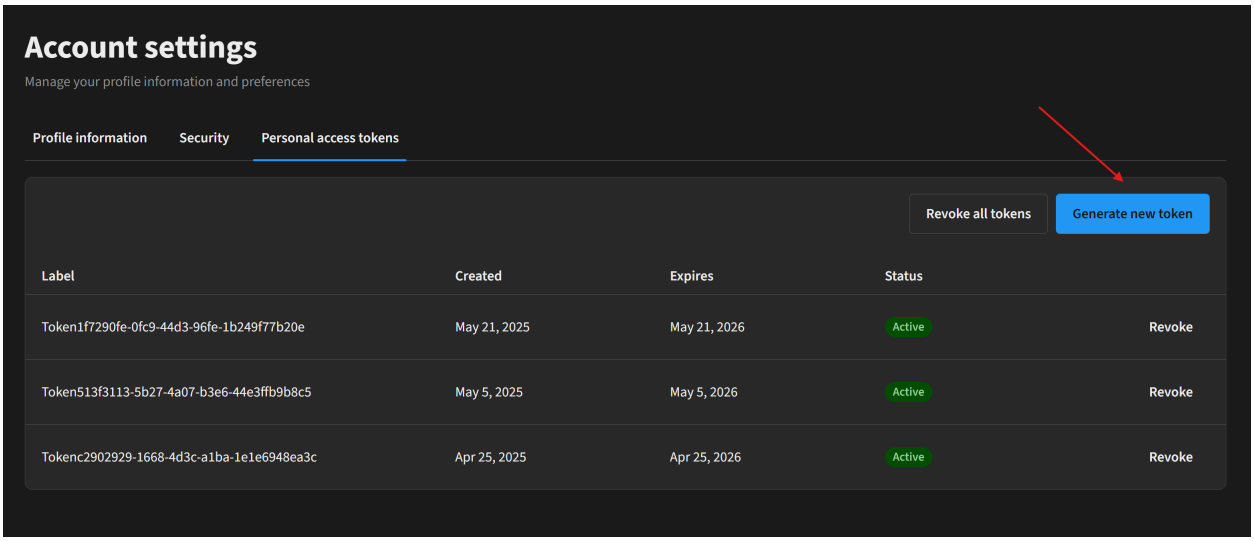
Old Method (For Older Server Versions)

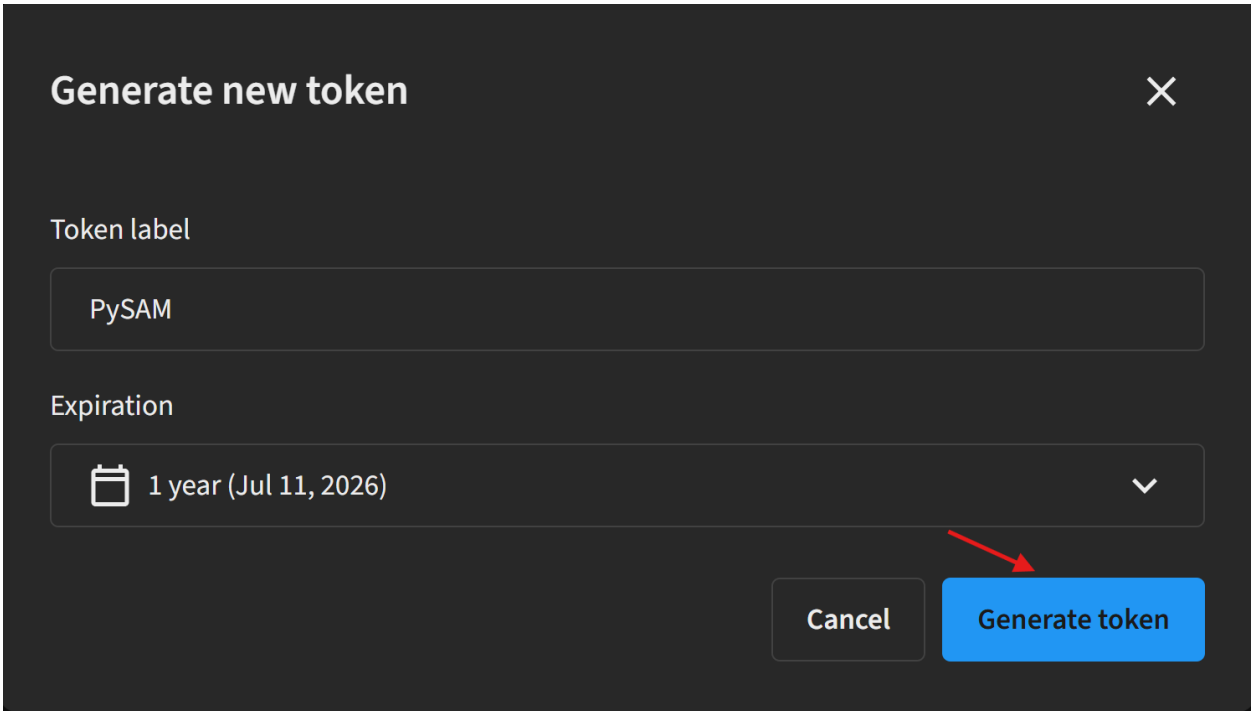
If you are using an older server version that does not support the direct account settings method, you can generate a PAT via the Swagger UI.

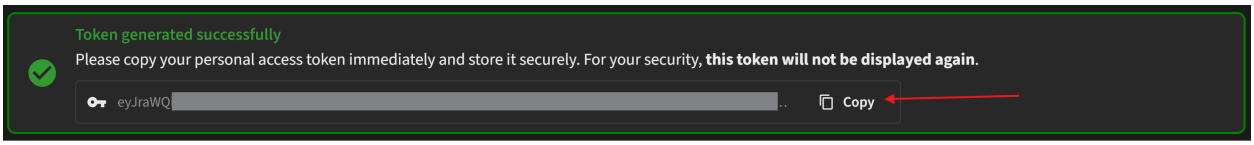
1. **Obtain a Temporary Bearer Token:** First, you need a valid Bearer Token. Follow the steps in the *Temporary Token* section to retrieve it using your browser's developer tools.


26.1.0-SNAPSHOT
TESTING
adrien.grzechowiak
adrien.grzechowiak@ansys.com
w all
Account settings
Change organization
first-orga
Theme
Sign out

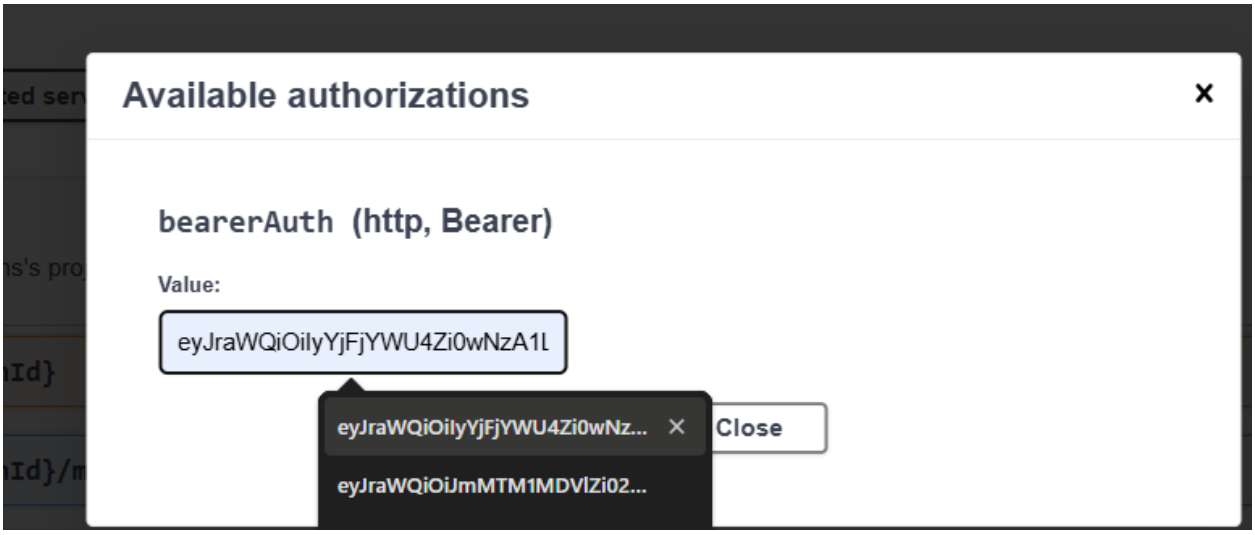

Account settings
Manage your profile information and preferences
Profile information
Security
Personal access tokens
Picture profile
Upload new picture
Image size must be at least 150px. Allowed file types: PNG, GIF, and JPG.


Account settings
Manage your profile information and preferences
Profile information
Security
Personal access tokens
Revoke all tokens
Generate new token
Label
Created
Expires
Status
Token1f7290fe-0fc9-44d3-96fe-1b249f77b20e
May 21, 2025
May 21, 2026
Active
Revoke
Token513f3113-5b27-4a07-b3e6-44e3ffb9b8c5
May 5, 2025
May 5, 2026
Active
Revoke
Tokenc2902929-1668-4d3c-a1ba-1e1e6948ea3c
Apr 25, 2025
Apr 25, 2026
Active
Revoke

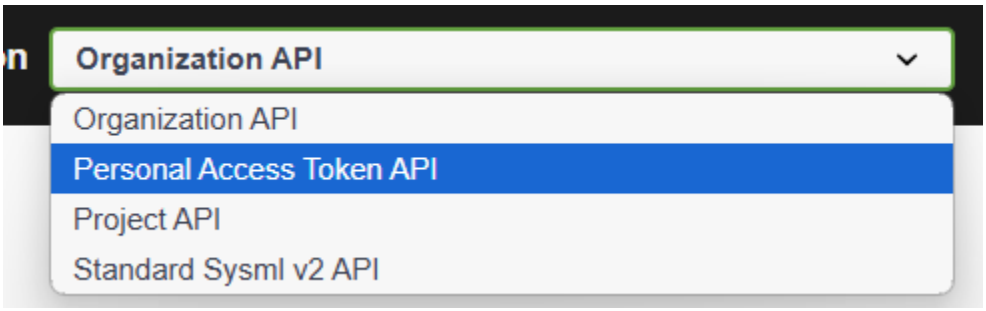

Generate new token
Token label
PySAM
Expiration
1 year (Jul 11, 2026)
Cancel
Generate token


Token generated successfully
Please copy your personal access token immediately and store it securely. For your security, this token will not be displayed again.
eyJraWQ
Copy

2. **Access Swagger UI:** Once you have your Bearer Token, navigate to the *Swagger UI* page. For example: *https://<SERVER-URL>/api/swagger-ui/index.html*
3. **Authorize with Bearer Token:** Click the **Authorize** button and paste your Bearer Token into the authentication field.



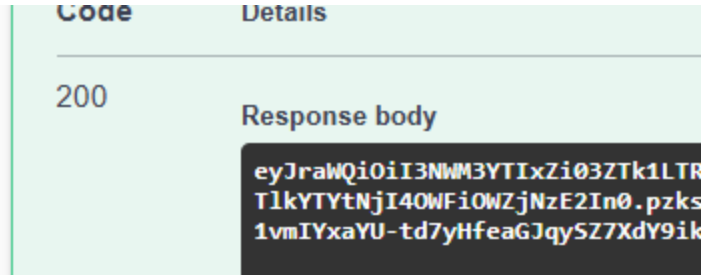
4. **Select Personal Access Token API:** Once authorized, go to the top right of the page and open the *Select a definition* dropdown. Scroll down and select the **Personal Access Token API**.



5. **Generate PAT via POST Request:** Use the **POST** request to generate a new PAT. You can optionally define the number of days the token should remain valid (the default is 30 days).

   After submitting the request, your newly generated PAT appears in the response body.

> **Note**
>
> Make sure to save your PAT somewhere secure, as it is not displayed again once you leave the page. You can always generate a new one if needed.


Code
Details
200
Response body
eyJraWQiOiI3NWM3YTIxZi03ZTk1LTR
TlkYTYtNjI4OWFiOWZjNzE2In0.pzks
1vmIYxaYU-td7yHfeaGJqySZ7XdY9ik

CHAPTER

THREE

DOCUMENTATION

3.1 Connector package

Note

To find all required data (Ids, token,..) see *Here*

3.1.1 AnsysSysML2APIConnector

class ansys.sam.sysml2.api.ansys_sysml2_api_connector.**AnsysSysML2APIConnector**(*server_url: str, token: str, organization_id: str, use_ssl: bool = True*)

Bases: `TemplateSysML2APIConnector`

Specific connector class for Ansys SysML 2 Connector.

__init__(*server_url: str, token: str, organization_id: str, use_ssl: bool = True*)

Construct Method for new instance.

Parameters

server_url
[str] The server URL

token
[str] Authentication token

organization_id
[str] Project organization id

use_ssl
[bool, optional] if the server url SSL (valid HTTPS), by default True

create_project(*project_name: str, project_description: str = 'Project description'*) → dict

Create a project with the associated name and description.

Parameters

project_name
[str] name of the project

project_description (optional)
[str] description of the project

Returns

dict
Project record

`execute_query`(*project_id: str*, *query: str*) → dict

Send a query to the Standard API using the connector.

Parameters

project_id
[str] Project id

query
[str] The query, in JSON format

Returns

dict
Result of the query

`get_all_elements`(*project_id: str*) → list

Return all elements of the given project.

Parameters

project_id
[str] Project Id

Returns

list
The list of all elements

`get_element_by_id`(*project_id: str*, *element_id: str*) → dict

Return information of the given element.

Parameters

project_id
[str] Id of the project where the element is

element_id
[str] Id of the wanted element

Returns

dict
Information of the element

`get_project_by_id`(*project_id: str*) → dict

Return information for the given project.

Parameters

project_id
[str] Id of the project

Returns

dict
Information of the project

get_projects() → list

Return all projects of the connected user.

Returns

list
The list of all projects

get_root_elements(*project_id: str*) → list

Return all root elements of the project.

Parameters

project_id
[str] Project id

Returns

list
All root elements

3.1.2 SamRestApiConnector

class ansys.sam.sysml2.diagrams.api.sam_rest_api_connector.**SamRestApiConnector**(*server_url: str, token: str, use_ssl: bool = True*)

SAM REST API Connector implementation.

get_all_diagram_image_from_project(*project_id: str, file_format: str*) → bytes

Download all diagrams from a project as a compressed ZIP archive.

get_diagram_image_as_jpeg(*project_id: str, diagram_id: str*) → bytes

Download a diagram rendered as JPEG format.

get_diagram_image_as_png(*project_id: str, diagram_id: str*) → bytes

Download a diagram rendered as PNG format.

get_diagram_image_as_svg(*project_id: str, diagram_id: str*) → bytes

Download a diagram rendered as SVG format.

get_diagrams_info(*project_id: str*) → dict

Fetch metadata and information for all diagrams within a specific project.

get_project_data(*model_id: str*) → dict

Retrieve project data from the SAM API using the project identifier.

get_single_diagram_info(*project_id: str, diagram_id: str*) → dict

Retrieve detailed information for a single diagram within a project.

3.2 Diagrams utilities package

3.2.1 SamDiagramDownloader

class ansys.sam.sysml2.diagrams.tools.sam_diagram_downloader.**SamDiagramDownloader**(*connector: SamApiConnector, project_id: str*)

Bases: object

Class to manage and download diagrams.

download_all_diagrams(*path: str | Path, file_format: str = 'svg', filename: str = ''*) → str

Download all diagrams as a ZIP archive.

Parameters

path
[str or Path] Destination directory or file path for the ZIP file.

file_format
[str, optional] Format of the diagrams (e.g., 'png', 'jpeg', 'svg'). Default is "svg".

filename
[str, optional] Name of the file. Default is "{Project Name}_{Image Extension}_diagrams.zip".

Returns

str
Path to the saved ZIP file.

Raises

DiagramConnectorException
If the download fails or HTTP response is not 200.

download_diagram(*diagram_id: str, path: str | Path, file_format: str = 'svg', filename: str = ''*) → str

Download a single diagram and save it to disk.

Parameters

diagram_id
[str] The diagram id to download.

path
[str or Path] Destination file or directory.

file_format
[str] Format of the diagram (e.g., 'svg', 'png', 'jpeg').

filename
[str, optional] Name of the file. Default is diagram ID.

Returns

str
Return the path of the file created.

Raises

DiagramNotAvailableException
If diagram functionality is not available for this project.

DiagramConnectorException
If the download fails or HTTP response is not 200.

3.3 AnsysSysML2Project

class ansys.sam.sysml2.tools.ansys_sysml2_project.**AnsysSysML2Project**(*server_url: str, token: str, organization_id: str, project_id: str, use_ssl: bool = True*)

Bases: `ProjectImpl`

Complete Ansys SysML2 project implementation with integrated capabilities.

Parameters

server_url
[`str`] Base URL of the SysML2 server.

token
[`str`] Authentication token for API access.

organization_id
[`str`] Unique identifier of the organization.

project_id
[`str`] Unique identifier of the project to manage.

use_ssl
[bool, `optional`] Whether to use SSL/TLS for connections. Default is True.

__init__(*server_url: str, token: str, organization_id: str, project_id: str, use_ssl: bool = True*) → None

Initialize the AnsysSysML2Project with connection parameters.

Parameters

server_url
[`str`] Base URL of the SysML2 server.

token
[`str`] Authentication token for API access.

organization_id
[`str`] Unique identifier of the organization.

project_id
[`str`] Unique identifier of the project to manage.

use_ssl
[bool, `optional`] Whether to use SSL/TLS for connections. Default is True.

Raises

`ConnectionError`
If unable to establish connection to the server.

`AuthenticationError`
If the provided token is invalid.

`ProjectNotFoundError`
If the specified project does not exist.

create_element(*element_type: str, **kwargs*) → *SysMLElement*

Create a new SysML element in the project.

Parameters

element_type
[str] Type of the SysML element to create.

****kwargs**
Additional parameters for element creation.

Returns

SysMLElement
The newly created SysML element instance.

Raises

ValueError
If element_type is not supported or required parameters are missing.

ConnectionError
If unable to commit the new element to the server.

download_all_diagrams(*path: str | Path, file_format: str = 'svg', filename: str = ''*) → str

Download all diagrams as a ZIP archive.

Parameters

path
[str or Path] Destination directory or file path for the ZIP file.

file_format
[str, optional] Format of the diagrams (e.g., 'png', 'jpeg', 'svg'). Default is "svg".

filename
[str, optional] Name of the file. Default is "{Project Name}_{Image Extension}_diagrams.zip".

Returns

str
Path to the saved ZIP file.

Raises

DiagramNotAvailableException
If diagram functionality is not available for this project.

DiagramConnectorException
If the download fails or HTTP response is not 200.

download_diagram(*diagram_id: str, path: str | Path, file_format: str = 'svg', filename: str = ''*) → str

Download a single diagram and save it to disk.

Parameters

diagram_id
[str] The diagram id to download.

path
[str or Path] Destination file or directory.

file_format
[str] Format of the diagram (e.g., 'svg', 'png', 'jpeg').

filename
[str, optional] Name of the file. Default is diagram ID.

Returns

str
Return the path of the file created.

Raises

DiagramNotAvailableException
If diagram functionality is not available for this project.

DiagramConnectorException
If the download fails or HTTP response is not 200.

get_project_diagrams_info() → dict

Get information about available diagrams in the project.

Returns

dict
Dictionary containing diagram information.

Raises

DiagramNotAvailableException
If diagram functionality is not available for this project.

get_single_diagram_info(*diagram_id: str*) → dict

Get information about a diagram in the project.

Parameters

diagram_id: str
ID of the diagram we want info about.

Returns

dict
Dictionary containing diagram information.

Raises

DiagramNotAvailableException
If diagram functionality is not available for this project.

is_diagrams_available() → bool

Check if diagram functionality is available for this project.

Returns

bool
True if diagrams can be downloaded, False otherwise.

3.4 Diagrams package

3.4.1 SAMDiagramManager

class ansys.sam.sysml2.diagrams.sam_diagram_manager.**SAMDiagramManager**(*connector: SamApiConnector*)

Bases: object

Diagram Manager class.

load_diagrams(*model: Project*) → None

Load diagrams into a model.

3.5 Creation package

3.5.1 Factory

class ansys.sam.sysml2.tools.factory.**Factory**(*project:* Project, *conn:* AnsysSysML2APIConnector)

Bases: object

Python Factory class for creating new SysMLElements.

__init__(*project:* Project, *conn:* AnsysSysML2APIConnector) → None

Initialize Factory class.

create_element(*element_type:* *str*, ***kwargs*)

Create a new element in the model and return it to the User.

Parameters

element_type
[str] Type of the element

kwargs
[Any] Other parameters of the new element

Returns

SysMLElement
The Created Element

3.6 Project manager

3.6.1 SysML2ProjectManager

class ansys.sam.sysml2.builder.sysml2_project_manager.**SysML2ProjectManager**(*connector: SysML2APIConnector*)

Bases: object

Director class for project load.

__init__(*connector: SysML2APIConnector*)

Construct Method for new instance.

Parameters

connector
[SysML2APIConnector] The SysML 2 connector.

get_project(*project_id:* *str*) → *Project*

Getter to load a project from API in Python Object.

Parameters

project_id
[str] The project ID

Returns

Project
Mapped Project.

3.7 Loaded project

3.7.1 Project

class ansys.sam.sysml2.classes.project.**Project**

Project Interface for users.

find_element_by_id(*element_id: str*) → *SysMLElement*

Find element with id.

Parameters

element_id
[str] Element Id

Returns

SysMLElement
The list of elements retrieved

find_elements_by_name(*elements_name: str*) → List[*SysMLElement*]

Find all element with name.

Parameters

elements_name
[str] Name if elements

Returns

List[SysMLElement]
The list of elements retrieved

get_root() → List[*SysMLElement*]

Return the list of root packages.

Returns

List[SysMLElement]
List of roots elements

get_root_package() → *SysMLElement*

Return the root package.

3.8 Classes package

3.8.1 SysMLElement

class ansys.sam.sysml2.classes.sysml_element.**SysMLElement**(*id: str*)

Bases: object

Python Base class for all SysML Element.

get_value()

Return the value of the feature.

parse_and_set_value(*value: str*)

Parse the value and create the valuation part in the Feature.

set_value(*new_value: str | int | float | bool*)

Update the Feature value.

3.9 Tools package

3.9.1 SysMLTools

class ansys.sam.sysml2.tools.sysmltools.**SysMLTools**

Bases: object

Static class for all sysml tools.

static isinstance(*element*, *type: str*) → bool

Use this function to check type of an element.

Parameters

element
[SysMLElement] The element to check

type
[str] SysML class name

Returns

bool
True if yes, False else

CHAPTER

FOUR

EXAMPLES

This section shows different ways to use the API with some examples.

4.1 Bike example

First of all, a bike model is required to work on.

Fortunately, one is provided for you.

Download this model : `Bike Model`.

Open SAM Editor on your browser, and select the wanted organization (*MyOrga* for example). Then, click **New Project > SysML V2 > Import File**. Click on **Choose File** in the **File to import** input, and select the `bike.xmi` file you just downloaded. The name of the project is automatically set to `bike`. Click on **Import** and wait for the project to be loaded.

Congratulations, you now have a bike model to work on

4.1.1 Calculate the bike weight

Calculate the weight of the bike.

When looking at the model, the weight of the bike is the sum of the weight of the frame and the weight of all elements of the wheel components.

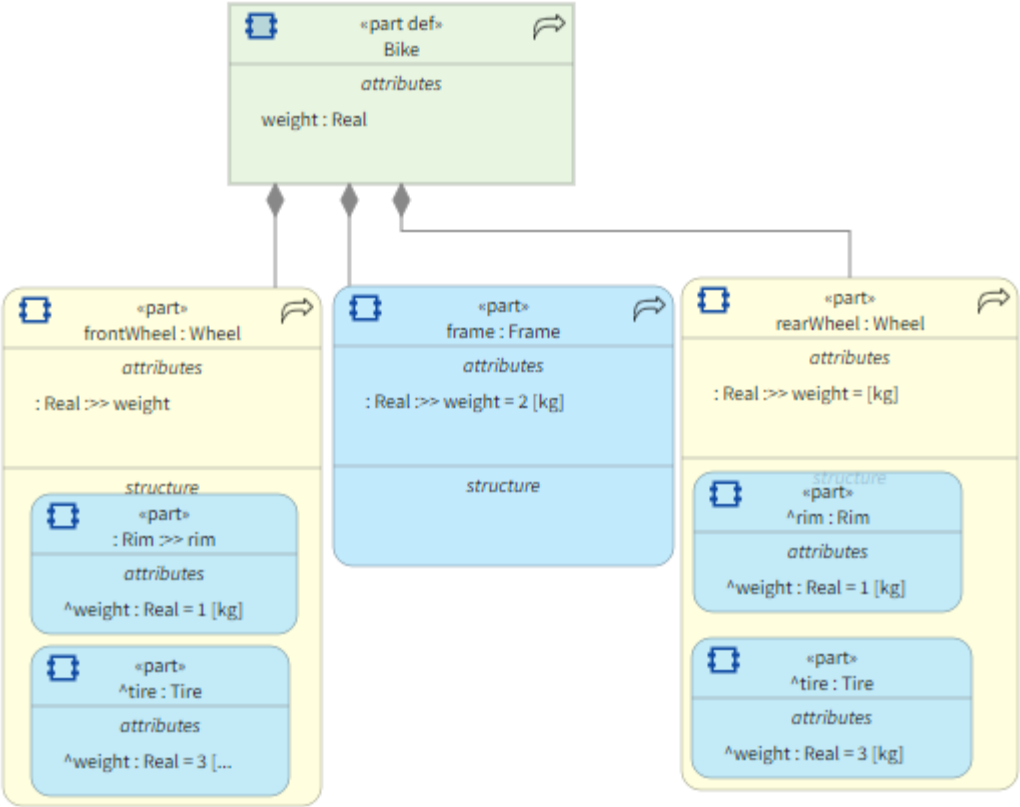
> **Note**
>
> In this case, calculate the sum of all blue elements of the model:

Step 1: Load the project

> **Note**
>
> This assumes that you have already installed the Library. If not, please refer to the *Installation* section.

Before loading the project, create a Connector and a Project Manager.

See section *Organization Id* and *Bearer Token* to know how to get the required data.


«part def»
Bike
attributes
weight : Real
«part»
frontWheel : Wheel
attributes
: Real :>> weight
structure
«part»
: Rim :>> rim
attributes
^weight : Real = 1 [kg]
«part»
^tire : Tire
attributes
^weight : Real = 3 [...
«part»
frame : Frame
attributes
: Real :>> weight = 2 [kg]
structure
«part»
rearWheel : Wheel
attributes
: Real :>> weight = [kg]
«part»
^rim : Rim
attributes
^weight : Real = 1 [kg]
«part»
^tire : Tire
attributes
^weight : Real = 3 [kg]

Listing 1: Import libraries and create connection to SysML2 API server

```
# Import Connector and Model Manager
import requests
from urllib3.exceptions import InsecureRequestWarning

from ansys.sam.sysml2 import AnsysSysML2APIConnector, SysML2ProjectManager

# Used to disable warnings
requests.packages.urllib3.disable_warnings(InsecureRequestWarning)

# Create your connector for the Sam Server
ansyssysml2apiconnector = AnsysSysML2APIConnector(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    organization_id="<Orga ID>",  # The Organization ID
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # If the server has a valid SSL
)
```

Now, that you are logged in, you can load the project `bike`.

> **Note**
>
> To load a project, its ID is needed. You can find it in the URL of the Editor.

Listing 2: Get the bike project

```
my_bike_project = project_manager.get_project("<Bike Project ID>")
```

`myBikeProject` is the project to work on. See the *loaded project* section for more details about the project object.

Step 2: Calculate the weight

Now that the project has been loaded, calculate the weight of the bike.

Get the Bike element.

> **Note**
>
> There are many ways to get an element, here the dot notation is used. See the *Getter* section for more details.

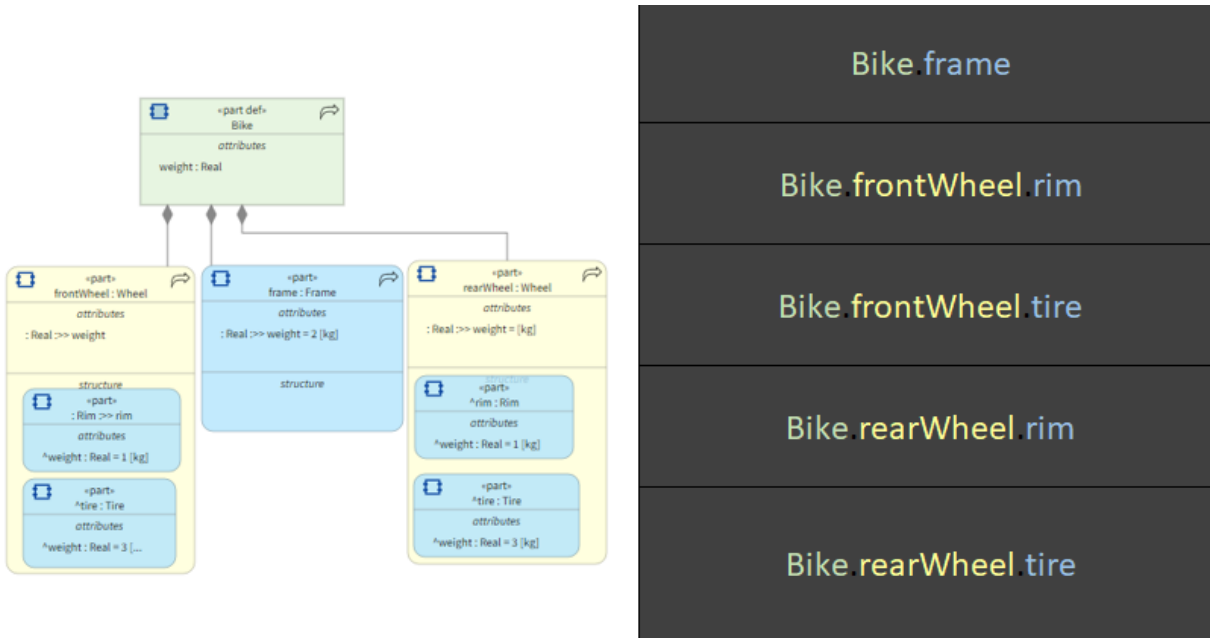
Listing 3: Get the bike part definition from the project structure

```
bike = my_bike_project.get_root_package().Structure.Bike
```

For weight calculation, nothing more simpler than simple addition.

So, if looking at each piece of the bike:

And to get the weight of each piece, just use the `weight` attribute, with the dot notation.



Listing 4: Calculate total bike weight by summing component weights

```
bike_weight = (
    bike.frontWheel.rim.weight.get_value()[0]
    + bike.frontWheel.tire.weight.get_value()[0]
    + bike.rearWheel.rim.weight.get_value()[0]
    + bike.rearWheel.tire.weight.get_value()[0]
    + bike.frame.weight.get_value()[0]
)
```

And print the weight of the bike.

Listing 5: Print the calculated bike weight

```
print(bike_weight)
```

You now have the total weight of the bike.

> **Note**
>
> You can get the full code `Here`.

4.2 Computer example

A computer model is required to get started.

One is provided for your convenience.

Download this model : `Computer Model`.

Open SAM Editor on your browser, and select the wanted organization (*MyOrga* for example). Then, click **New Project** > **SysML V2** > **Import File**. Click on **Choose File** in the **File to import** input, and select the `computer.xmi` file you

just downloaded. The name of the project is automatically set to `computer`. Click on *Import* and wait for the project to be loaded.

Congratulations, you now have a computer model to work on !

4.2.1 Calculate the computer cost

> **Note**
>
> You need to change *Organization ID*, *Server URL* and *Token* with your own data, see *this section for more information*.

Listing 6: Assessing system cost using recursive traversal

```python
"""Computer Cost Example for PySam."""

import requests
from urllib3.exceptions import InsecureRequestWarning

from ansys.sam.sysml2 import AnsysSysML2APIConnector, SysML2ProjectManager
from ansys.sam.sysml2.tools import SysMLTools

# Used to disable warnings
requests.packages.urllib3.disable_warnings(InsecureRequestWarning)

ansyssysml2apiconnector = AnsysSysML2APIConnector(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    organization_id="<Orga ID>",  # The Organization ID
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # If the server has a valid SSL
)

project_manager = SysML2ProjectManager(connector=ansyssysml2apiconnector)

project = project_manager.get_project(
    "<Computer Project ID>"
)  # You can find it in the URL of the Editor

real_systems = project.get_root_package().RealSystems


def assess_cost(element):
    """Calculate the cost of element."""
    if hasattr(element, "cost") and (element.cost.get_value() is not None):
        return element.cost.get_value()
    cost = 0
    for sub_element in element._ownedElement:
        if SysMLTools.isinstance(sub_element, "PartUsage"):
            cost += assess_cost(sub_element)
    return cost


```

(continues on next page)

(continued from previous page)

```
for system in real_systems._ownedElement:
    print(system._name, " : ", assess_cost(system))
```

4.3 Creating new element example

Make sure you have access to a valid server and a project containing the `Bike` structure.

If not, you can download this model : `Bike Model`.

Open SAM Editor on your browser, and select the wanted organization (*MyOrga* for example). Then, click **New Project > SysML V2 > Import File**. Click on **Choose File** in the **File to import** input, and select the `bike.xmi` file you just downloaded. The name of the project is automatically set to `bike`. Click on **Import** and wait for the project to be loaded.

Congratulations, you now have a bike model to work on !

4.3.1 Create an attribute usage for the bicycle frame length

This example shows how to create a new `AttributeUsage` element inside the `Bike` and assign it a value.

> **Note**
>
> You need to change *Organization ID*, *Server URL* and *Token* with your own data, see *this section for more information*.

Listing 7: Creating a new AttributeUsage element using the Factory class

```
"""Creating Element Example for PySam."""

import requests
from urllib3.exceptions import InsecureRequestWarning

from ansys.sam.sysml2 import AnsysSysML2APIConnector, SysML2ProjectManager
from ansys.sam.sysml2.tools import Factory

# Used to disable warnings
requests.packages.urllib3.disable_warnings(InsecureRequestWarning)

# Create your connector for the Sam Server
ansyssysml2apiconnector = AnsysSysML2APIConnector(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    organization_id="<Orga ID>",  # The Organization ID
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # IF the server has a valid SSL
)

project_manager = SysML2ProjectManager(connector=ansyssysml2apiconnector)
project = project_manager.get_project("<Bike Project ID>")

bike = project.get_root_package().Structure.Bike

factory = Factory(project, ansyssysml2apiconnector)
```

(continues on next page)

(continued from previous page)

```

new_bicycle_frame_length = factory.create_elements(
    element_type="AttributeUsage", name="length", owner=bike.frame
)

new_bicycle_frame_length.parse_and_set_value("60 [cm]")

print(project.get_root_package().Structure.Bike.frame.length.get_value())
```

You just created a new element and assigned a parsed value to it.

Note

You can also assign a value directly when creating the element, without using `set_value` or `parse_and_set_value`. There are two ways:

- Use `value=...` for simple values (e.g., numbers).
- Use `expression="..."` for values with units or expressions.

```
new_bicycle_frame_length_with_value = factory.create_element(
    element_type="AttributeUsage",
    name="lengthWithValue",
    owner=bike.frame,
    value=60
)

new_bicycle_frame_length_with_expression = factory.create_element(
    element_type="AttributeUsage",
    name="lengthWithExpression",
    owner=bike.frame,
    expression="60 [cm]"
)
```

This lets you set values directly at creation time, depending on your data format.

4.4 Download diagrams and create new elements

This example demonstrates how to work with the *Bike* model in a SysML v2 project using the Ansys SAM API. It covers:

- Loading and downloading diagrams.
- Navigating and saving diagram content.

Note

If you have never used the PySAM library before, it is recommended to begin with a simpler example to understand how the library works (*bike_weight*, *computer_cost*).

4.4.1 Prerequisites

Ensure you have:

1. A running SAM server instance.
2. A valid **Organization ID**, **Project ID**, and **Token**.
3. The *bike.xmi* model imported into your project.

4.4.2 Python example

Listing 8: Example of working with diagrams using SAMDiagramManager

```python
"""Download Diagrams Example for PySam."""

import requests
from urllib3.exceptions import InsecureRequestWarning

from ansys.sam.sysml2 import AnsysSysML2APIConnector, SysML2ProjectManager
from ansys.sam.sysml2.diagrams.api.sam_rest_api_connector import SamRestApiConnector
from ansys.sam.sysml2.diagrams.sam_diagram_manager import SAMDiagramManager
from ansys.sam.sysml2.diagrams.tools.sam_diagram_downloader import SamDiagramDownloader

# Used to disable warnings
requests.packages.urllib3.disable_warnings(InsecureRequestWarning)

ansyssysml2apiconnector = AnsysSysML2APIConnector(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    organization_id="<Orga ID>",  # The Organization ID
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # If the server has a valid SSL
)

sam_rest_api_connector = SamRestApiConnector(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # If the server has a valid SSL
)

project_manager = SysML2ProjectManager(connector=ansyssysml2apiconnector)

project = project_manager.get_project("<Bike Project ID>")

# ----------------------------------------
# Work with Diagrams
# ----------------------------------------

with SAMDiagramManager(connector=sam_rest_api_connector) as diagrams:
    diagrams.load_diagrams(model=project)

print(f"Loaded {len(project.get_root_package().__diagram)} diagrams.")

downloader = SamDiagramDownloader(connector=sam_rest_api_connector, project_id=project._
```

(continues on next page)

(continued from previous page)

```
↪id)

# ----------------------------------------
# Download Diagrams
# ----------------------------------------

### You can specify the file_format and the filename,
### But as default: file_format="svg", filename="<PackageName>_<FileFormat>_diagrams.zip"
path = downloader.download_all_diagrams(
    path="C:/Diagrams/Images/",
    file_format="jpeg",
    filename="download_all_diagrams_with_args.zip",
)
print(f"ZIP saved at: {path}\n")


first_diagram = project.get_root_package().__diagram[0]
first_diagram_id = first_diagram._id

path = downloader.download_diagram(
    diagram_id=first_diagram_id,
    path="C:/Diagrams/Images/",
    file_format="svg",
)
print(f"Diagram saved at: {path}")


usage_diagrams = project.get_root_package().Usage.__diagram
for i, diagram in enumerate(usage_diagrams, 1):
    downloader.download_diagram(
        diagram_id=diagram._id, file_format="png", path="C:/Diagrams/Images/Usage"
    )
    print(f"> Saved Usage diagram #{i}: {diagram._plane._model_element._name}")

# ----------------------------------------
# Navigate through Diagrams
# ----------------------------------------

print(first_diagram._plane._model_element._name)

for diagram in usage_diagrams:
    print("Diagram name:", diagram._name)
```

Note

- Replace placeholder values with your actual SAM configuration.
- Always use `SAMDiagramManager` within a context manager (`with` statement).
- Diagram content can be retrieved in various formats: `png`, `svg`, etc.

4.5 Simplified SAM project initialization

This example demonstrates how to use the simplified *AnsysSysML2Project* class to work with a SysML v2 project on SAM. This approach reduces boilerplate code by automatically initializing all necessary connectors. It covers:

- Simplified project initialization with a single class.
- Downloading diagrams (single and batch).
- Creating new elements.
- Navigating through project diagrams.
- Retrieving diagram information.

> **Note**
>
> The `AnsysSysML2Project` class is specifically designed for SAM projects and automatically handles the initialization of `AnsysSysML2APIConnector`, `SamRestApiConnector`, `SysML2ProjectManager`, and `SAMDiagramManager`.

4.5.1 Prerequisites for simplified SAM

Ensure you have:

1. A running SAM server instance.
2. A valid **Organization ID**, **Project ID**, and **Token**.
3. The *bike.xmi* model imported into your project.

4.5.2 Simplified python example

Listing 9: Simplified example using AnsysSysML2Project for SAM projects

```python
"""AnsysSysML2Project Example for PySam."""

import requests
from urllib3.exceptions import InsecureRequestWarning

from ansys.sam.sysml2.tools.ansys_sysml2_project import AnsysSysML2Project

requests.packages.urllib3.disable_warnings(InsecureRequestWarning)

project = AnsysSysML2Project(
    server_url="https://127.0.0.1:8443/",  # Your Sam server base URL
    organization_id="<Orga ID>",  # The Organization ID
    token="<Token>",  # Your Auth Token (See section below)
    use_ssl=False,  # IF the server has a valid SSL
    project_id="<Bike Project ID>",  # The Project ID
)

diagrams_status = project.is_diagrams_available()
print(f"Diagrams Status : {'Available' if diagrams_status else 'Unavailable'}", end="\n\n
↪")
```

(continues on next page)

(continued from previous page)

```python

SAVE_IMAGE_PATH = "C:/Diagrams/Images/"
first_diagram = project.get_root_package().__diagram[0]
first_diagram_id = first_diagram._id

# ----------------------------------------
# Download ZIP file containing Diagrams
# ----------------------------------------

response = project.download_all_diagrams(path=SAVE_IMAGE_PATH, file_format="png",␣
↪filename="png")
print(f"> ZIP saved at: {response}\n")

# ----------------------------------------
# Download single Diagram
# ----------------------------------------

path = project.download_diagram(
    diagram_id=first_diagram_id,
    filename="CUSTOM_FILENAME",
    file_format="png",
    path=SAVE_IMAGE_PATH,
)
print(f"> Diagram saved at: {path}\n")

usage_diagrams = project.get_root_package().Usage.__diagram
for i, diagram in enumerate(usage_diagrams, 1):
    project.download_diagram(
        diagram_id=diagram._id, file_format="png", path="C:/Diagrams/Images/Usage"
    )
    print(f"> Saved Usage diagram #{i}: {diagram._plane._model_element._name}\n")

# ----------------------------------------
# Navigate through Diagrams
# ----------------------------------------

print(first_diagram._plane._model_element._name, end="\n")

for diagram in project.get_root_package().Usage.__diagram:
    print("Diagram name:", diagram._name, end="\n")

# ----------------------------------------
# Create Element
# ----------------------------------------

project.create_element(element_type="AttributeUsage", name="NewAttr")

# ----------------------------------------
# Get Diagrams Info
# ----------------------------------------

print(project.get_project_diagrams_info(), end="\n\n")
```

(continues on next page)

(continued from previous page)

```
print(project.get_single_diagram_info(first_diagram_id))
```

4.5.3 Key advantages

Compared to the traditional approach (*download_diagrams*), this simplified method offers:

- **Single initialization**: One class handles all connectors automatically.
- **Built-in diagram management**: No need to manually create and manage `SAMDiagramManager`.
- **Streamlined API**: Direct access to project operations without managing multiple connector instances.
- **Integrated feature**: Combines project management, diagram operations, and element creation in one interface.

Note

- Replace placeholder values with your actual SAM configuration.
- The `AnsysSysML2Project` class automatically manages diagram loading and connector lifecycle.
- All diagram formats (`png`, `svg`, `jpeg`) are supported for downloads.

CHAPTER

FIVE

RELEASE NOTES

This document contains the release notes for the project.